

3. PRACTICE_CODE

1. Mobile Class with Brand and Price

```
class Mobile:
    def __init__(self, brand, price):
        self.brand = brand
        self.price = price

    def display(self):
        print(f"Brand: {self.brand}")
        print(f"Price: ₹{self.price}")

m1 = Mobile("Samsung", 25000)
m1.display()
```

2. Laptop Class and Display Specifications

```
class Laptop:
    def __init__(self, brand, ram, processor):
        self.brand = brand
        self.ram = ram
        self.processor = processor

    def display(self):
        print("Brand:", self.brand)
        print("RAM:", self.ram)
        print("Processor:", self.processor)
```

```
l1=Laptop("Dell","16GB","Intel i7")  
l1.display()
```

3. Area of Triangle Using Class

```
classTriangle:  
    def__init__(self,base,height):  
        self.base=base  
        self.height=height  
  
    defarea(self):  
        return0.5*self.base*self.height  
  
t=Triangle(10,5)  
print("Area:",t.area())
```

4. Movie Class

```
classMovie:  
    def__init__(self,title,director,year):  
        self.title=title  
        self.director=director  
        self.year=year  
  
    defdisplay(self):  
        print(self.title,self.director,self.year)  
  
m=Movie("3 Idiots","Rajkumar Hirani",2009)  
m.display()
```

5. Vehicle and Bike (Inheritance)

```
class Vehicle:
    def start(self):
        print("Vehicle Started")

class Bike(Vehicle):
    def ride(self):
        print("Bike is Riding")

b=Bike()
b.start()
b.ride()
```

6. SavingsAccount Inheriting BankAccount

```
class BankAccount:
    def __init__(self, balance):
        self.balance=balance

class SavingsAccount(BankAccount):
    def add_interest(self):
        self.balance+=self.balance*0.05

acc=SavingsAccount(10000)
acc.add_interest()
print("Balance:", acc.balance)
```

7. Method Overriding (Employee and Manager)

```
classEmployee:
defrole(self):
print("I am an Employee")

classManager(Employee):
defrole(self):
print("I am a Manager")

e=Employee()
m=Manager()

e.role()
m.role()
```

8. Shopping Cart

```
classShoppingCart:
def__init__(self):
self.items= []

defadd_item(self,item):
self.items.append(item)

defshow_items(self):
print("Cart Items:")
foriteminself.items:
print(item)

cart=ShoppingCart()

cart.add_item("Laptop")
cart.add_item("Mouse")
```

```
cart.show_items()
```

9. Hospital Management System

```
class Patient:
    def __init__(self, name, disease):
        self.name = name
        self.disease = disease

    def display(self):
        print(self.name, "-", self.disease)

p1 = Patient("Rahul", "Fever")
p2 = Patient("Amit", "Cold")

p1.display()
p2.display()
```

10. Student Result Management System

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def result(self):
        if self.marks >= 40:
            return "Pass"
        return "Fail"

s = Student("John", 70)
```

```
print("Name:", s.name)
print("Result:", s.result())
```

11. Railway Reservation System

```
class Train:
    def __init__(self):
        self.seats = 5

    def reserve(self):
        if self.seats > 0:
            self.seats -= 1
            print("Seat Reserved")
        else:
            print("No Seats Available")

t = Train()

t.reserve()
t.reserve()

print("Remaining Seats:", t.seats)
```

12. ATM Simulation

```
class ATM:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount <= self.balance:
```

```
self.balance-=amount
print("Withdrawal Successful")
else:
print("Insufficient Balance")

atm=ATM(5000)

atm.withdraw(1000)
print("Balance:",atm.balance)
```

13. Online Food Ordering System

```
classFoodOrder:
def__init__(self):
self.orders= []

defadd_food(self,item):
self.orders.append(item)

defshow_order(self):
print("Ordered Items:",self.orders)

order=FoodOrder()

order.add_food("Pizza")
order.add_food("Burger")

order.show_order()
```

14. E-Commerce Product Catalog

```
classProduct:
def__init__(self,name,price):
self.name=name
self.price=price

defdisplay(self):
print(self.name,"-",self.price)

p1=Product("Laptop",60000)
p2=Product("Phone",20000)

p1.display()
p2.display()
```

15. School Management System

```
classSchool:
def__init__(self):
self.students= []

defadd_student(self,name):
self.students.append(name)

defshow_students(self):
print(self.students)

s=School()

s.add_student("Rahul")
s.add_student("Amit")

s.show_students()
```

16. Hotel Booking System

```
classHotel:
    def__init__(self):
        self.rooms=3

    defbook_room(self):
        ifself.rooms>0:
            self.rooms-=1
            print("Room Booked")
        else:
            print("No Room Available")

h=Hotel()

h.book_room()
print("Available Rooms:",h.rooms)
```

17. Quiz Application

```
classQuiz:
    def__init__(self,question,answer):
        self.question=question
        self.answer=answer

    defcheck(self,user_answer):
        returnuser_answer==self.answer

q=Quiz("Capital of India?","Delhi")

print(q.check("Delhi"))
```

18. Payroll System

```
classEmployee:
def__init__(self,name,salary):
self.name=name
self.salary=salary

defcalculate_salary(self):
returnself.salary

e=Employee("Rahul",50000)

print("Salary:",e.calculate_salary())
```

19. Parking Management System

```
classParking:
def__init__(self):
self.slots=10

defpark_vehicle(self):
ifself.slots>0:
self.slots-=1
print("Vehicle Parked")
else:
print("Parking Full")

p=Parking()

p.park_vehicle()

print("Available Slots:",p.slots)
```

20. Cricket Team Management System

```
class Player:
    def __init__(self, name, role):
        self.name = name
        self.role = role

    def display(self):
        print(self.name, "-", self.role)

class Team:
    def __init__(self):
        self.players = []

    def add_player(self, player):
        self.players.append(player)

    def show_team(self):
        for player in self.players:
            player.display()

team = Team()

team.add_player(Player("Virat Kohli", "Batsman"))
team.add_player(Player("Jasprit Bumrah", "Bowler"))

team.show_team()
```